

104. Les lutins en couleur

Le même personnage en rouge et en vert : deux joueurs pour une seule tuile.

1. Ce qu'il faut comprendre

Les lutins ont **huit palettes à eux**, réglées par `couleurLutin(palette, teinte, rouge, vert, bleu)`. Comme en quatre nuances, **l'indice 0 est transparent** : il ne reste que trois couleurs visibles par lutin — on règle donc les teintes 1, 2 et 3.

`teindreLutin(numero, palette)` choisit la palette d'un lutin. Ici, le même `HEROS` est dessiné deux fois : en palette 0 (rouge) et en palette 1 (vert) — un joueur et son frère, comme dans les jeux de plateforme.

Le piège : `sprite()` **remet la palette du lutin à 0** chaque fois qu'on l'appelle. Pour un lutin qui bouge — donc qu'on repose à chaque image —, il faut rappeler `teindreLutin()` **juste après** `sprite()`. Oublie-le, et le frère vert redevient rouge dès qu'il fait un pas.

Ici, le héros vert avance tout seul : son `teindreLutin` est dans la boucle, juste après son `sprite`.

Huit palettes de lutins, trois couleurs visibles chacune : **24 couleurs** pour les personnages, en plus des 32 du décor — **56** à l'écran en même temps, le plafond de la console.

À toi : ajoute un troisième héros en bleu, palette 2, qui avance dans l'autre sens.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <couleurLutin>` → [0.107.4. La fonction couleurLutin\(\) — une couleur des personnages](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <teindreLutin>` → [0.107.5. La fonction teindreLutin\(\) — un lutin dans une palette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **583 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage – la mise en route de la console
main()	196 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	67 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	128 o	← CopierTuiles – 7 lettres sur 43, et 1 tuile dessinée

```

DonneesTransfert    10 o ← InstallerTransfert  – la petite routine qui copie les lutins
"DEUX FRERES"      11 o ← texte()  – un texte gravé
Lettres gravées : DEFRSUX
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
couleurLutin() × 6    72 o sur place, dans main()
sprite()      × 2    46 o sur place, dans main()
teindreLutin() × 2    33 o sur place, dans main()
texte()       × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()       × 1     6 o sur place, dans main() + AttendreImage (28 o)
images()      × 1     2 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME (Game Boy Color) : deux héros du même dessin, l'un rouge et
// immobile, l'autre vert qui avance tout seul vers la droite.
//
// Ce qui est nouveau :
//  couleurLutin(paLETTE, indice, rouge, vert, bleu) : les couleurs des lutins
//    (8 palettes à eux). L'indice 0 est transparent : on règle 1, 2 et 3.
//  teindreLutin(numero du lutin, palette) : quelle palette ce lutin emploie.
//  LE PIÈGE : sprite() remet la palette du lutin à 0. Il faut donc rappeler
//  teindreLutin() juste APRÈS chaque sprite().

#include <Tuile>          // un dessin de 8 × 8 pixels
#include <couleurLutin>   // choisit une couleur d'une palette des lutins
#include <texte>          // écrit un texte à l'écran
#include <sprite>         // place un lutin de 8 × 8 au pixel près
#include <teindreLutin>   // met un lutin dans une palette

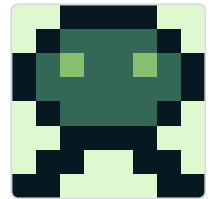
```

2. UN DESSIN — HEROS

```

Tuile HEROS = {          // 0 = transparent, 1 clair, 2 moyen, 3 contour.
    "00333300",
    "03222230",
    "32122123",
    "32222223",
    "03222230",
    "00333300",
    "03300330",
    "33000033",
};

```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    // Palette 0 des lutins : le rouge. Palette 1 : le vert.
    couleurLutin(0, 1, 31, 28, 20); // palette 0, indice 1 : rose clair
    couleurLutin(0, 2, 31, 4, 2); // palette 0, indice 2 : rouge
    couleurLutin(0, 3, 10, 0, 0); // palette 0, indice 3 : rouge très sombre
    couleurLutin(1, 1, 28, 31, 20); // palette 1, indice 1 : vert pâle
    couleurLutin(1, 2, 4, 26, 2); // palette 1, indice 2 : vert
    couleurLutin(1, 3, 0, 8, 0); // palette 1, indice 3 : vert très sombre

    texte(1, 1, "DEUX FRERES");

    sprite(0, 40, 70, HEROS);               // Lutin 0 au pixel (40, 70)...
    teindreLutin(0, 0);                     // ... en palette 0 : rouge. Il ne bougera plus.

    uint8_t x = 60;                         // La place du frère vert, en pixels.

    while (true) {                          // La boucle du jeu :
        image();

        if (images() % 4 == 0 && x < 150) x++; // Une image sur 4, et pas trop loin : + 1 pixel.
        sprite(1, x, 70, HEROS);             // Lutin 1, reposé à sa place à chaque image...
        teindreLutin(1, 1);                  // après CHAQUE sprite() : sinon il redevient rouge
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Deux petits personnages : un rouge immobile, un vert qui avance — et qui reste vert.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le premier est rouge (31/4/2)

✓ le second a bougé (x = 79)

✓ et il est toujours vert (4/26/2)

✓ le coin reste transparent

Ensuite : **105. Animer les couleurs : l'eau qui scintille** — chapitre 16 : La couleur : la Game Boy Color. · gameboy3 — écrire une cartouche Game Boy en C++